

算法原理 HW4 作业

姓名：冯峻 学号：523031910148

2025 年 11 月 7 日

目录

1. 题目：给定正整数集合 $S = \{a_1, a_2, \dots, a_n\}$ ，设计动态规划算法求解该集合的划分问题，即判断是否存在集合 $A \subseteq S$ ，使得 $\sum_{a_i \in A} a_i = \sum_{a_i \in S-A} a_i$ 。分析该问题的优化子结构、重叠子问题，给出动态规划算法伪代码并分析算法的复杂度。

解：

问题分析：

该问题等价于判断是否存在子集 A ，使得其元素之和等于集合总和的一半。设集合 S 的总和为 sum ，若 sum 为奇数，则显然无解；若 sum 为偶数，问题转化为：是否存在子集，其元素之和等于 $target = sum/2$ 。

优化子结构：

定义子问题： $dp[i][j]$ 表示使用前 i 个元素能否凑出和为 j 。对于第 i 个元素 a_i ，有两种选择：

- 不选 a_i ：问题规模缩减为前 $i-1$ 个元素能否凑出和为 j ，即 $dp[i][j] = dp[i-1][j]$
- 选 a_i ：问题规模缩减为前 $i-1$ 个元素能否凑出和为 $j - a_i$ ，即 $dp[i][j] = dp[i-1][j - a_i]$

因此，原问题的最优解包含子问题的最优解，具有**优化子结构**性质。

重叠子问题：

在计算 $dp[i][j]$ 时，可能需要多次计算相同的子问题 $dp[i-1][j']$ 。例如，计算 $dp[3][10]$ 和 $dp[3][12]$ 时，都可能需要 $dp[2][8]$ 的结果。这种重复计算的子问题体现了**重叠子问题**性质。

动态规划算法伪代码：

Algorithm SUBSET-PARTITION(S)

```

1:  $\text{sum} \leftarrow \sum_{i=1}^n S[i]$ 
2: if  $\text{sum} \bmod 2 \neq 0$  then
3:   return False
4:  $\text{target} \leftarrow \text{sum}/2$ 
5: 初始化二维数组  $dp[0..n][0..\text{target}]$ , 全部置为 False
6:  $dp[i][0] \leftarrow \text{True}, \forall i \in [0, n]$  ▷ 和为 0 总是可以达到
7: for  $i = 1$  to  $n$  do
8:   for  $j = 1$  to  $\text{target}$  do
9:      $dp[i][j] \leftarrow dp[i-1][j]$  ▷ 不选第  $i$  个元素
10:    if  $j \geq S[i]$  then
11:       $dp[i][j] \leftarrow dp[i][j] \vee dp[i-1][j-S[i]]$  ▷ 选第  $i$  个元素
12: return  $dp[n][\text{target}]$ 

```

时间复杂度分析:

算法需要填充一个 $n \times \text{sum}/2$ 的动态规划表。每个状态的计算时间为 $O(1)$ 。因此，总时间复杂度为 $O(n \cdot \text{sum}/2)$ 。

空间复杂度分析:

需要维护一个维度为 $n \times \text{sum}/2$ 的动态规划表，因此空间复杂度为 $O(n \cdot \text{sum}/2)$ 。

2. **题目：**课上讲过了子串和子序列的区别。现考虑两个字符串 X 和 Y ，求解其最长的公共子串。比如 $X = \text{'photograph'}$ ， $Y = \text{'tomography'}$ ，则其最长公共子串为 'ograph' 。

1) 已知 $n = |X|$ ， $m = |Y|$ ，请设计一个时间复杂度 $\Theta(nm)$ 的动态规划算法求解 X 和 Y 的最长公共子串。

2) 设计一个时间复杂度 $\Theta(nm)$ 的非动态规划算法求解 X 和 Y 的最长公共子串。

需给出相应的算法设计和分析过程，不能仅仅只给出伪代码。

解：

1) 动态规划算法

优化子结构分析：

定义状态 $dp[i][j]$ 表示以 $X[i-1]$ 和 $Y[j-1]$ **结尾** 的最长公共子串的长度。关键是“以... 结尾”这个限定条件。

状态转移方程：

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1, & \text{if } X[i-1] = Y[j-1] \\ 0, & \text{if } X[i-1] \neq Y[j-1] \end{cases}$$

当 $X[i-1] = Y[j-1]$ 时，以它们结尾的最长公共子串长度等于以 $X[i-2]$ 和 $Y[j-2]$ 结尾的最长公共子串长度加 1。当字符不相等时，由于子串必须连续，长度为 0。

这与最长公共子序列不同。在子序列问题中，当字符不相等时，状态转移为 $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$ ；而在最长公共子串问题中，由于连续性要求，状态直接置为 0。

重叠子问题：

在计算 $dp[i][j]$ 时，需要用到 $dp[i-1][j-1]$ 的结果。多个状态的计算会重复使用相同的子问题结果，体现了重叠子问题性质。

动态规划算法伪代码：

Algorithm LCS-SUBSTRING-DP(X, Y)

```

1:  $n \leftarrow |X|, m \leftarrow |Y|$ 
2: 初始化二维数组  $dp[0..n][0..m]$ , 全部置为 0
3:  $maxLength \leftarrow 0, endIndex \leftarrow 0$ 
4: for  $i = 1$  to  $n$  do
5:   for  $j = 1$  to  $m$  do
6:     if  $X[i - 1] = Y[j - 1]$  then
7:        $dp[i][j] \leftarrow dp[i - 1][j - 1] + 1$ 
8:       if  $dp[i][j] > maxLength$  then
9:          $maxLength \leftarrow dp[i][j]$ 
10:         $endIndex \leftarrow i - 1$  ▷ 记录在  $X$  中的结束位置
11:     else
12:        $dp[i][j] \leftarrow 0$ 
13: 从  $X[endIndex - maxLength + 1..endIndex]$  提取最长公共子串
14: return 最长公共子串及其长度  $maxLength$ 

```

时间复杂度：需要填充 $n \times m$ 的动态规划表，每个状态计算为 $O(1)$ ，因此总时间复杂度为 $\Theta(nm)$ 。

空间复杂度：需要维护一个维度为 $n \times m$ 的动态规划表，因此空间复杂度为 $\Theta(nm)$ 。

C++ 代码实现见：code/longest_common_substring_dp.cpp

2) 非动态规划算法

算法设计思路：

采用**对角线扫描法**，通过沿着字符串比较矩阵的所有对角线进行一次性扫描来寻找最长公共子串。

核心思想：

将两个字符串 X 和 Y 想象为一个 $n \times m$ 的比较矩阵，其中第 (i, j) 个位置表示 $X[i]$ 与 $Y[j]$ 的比较。最长公共子串对应于矩阵中某条对角线上的一段连续的匹配。

算法分两个阶段扫描所有可能的对角线：

- **阶段 1：**扫描从 X 的每个位置开始 ($i = 0, 1, \dots, n - 1$)， Y 从位置 0 开始的对角线，覆盖矩阵的下三角及主对角线部分
- **阶段 2：**扫描从 Y 的每个位置开始 ($j = 1, 2, \dots, m - 1$)， X 从位置 0 开始的对角线，覆盖矩阵的上三角部分

在每条对角线上，维护一个连续匹配长度计数器。遇到匹配字符时递增计数器，遇到不匹配字符时重置为 0（因为子串必须连续）。

算法正确性：

任意公共子串都对应于比较矩阵中某条对角线上的一段连续匹配。由于算法遍历了所有可能的对角线，因此一定能找到最长的公共子串。

非动态规划算法伪代码：

Algorithm LCS-SUBSTRING-DIAGONAL(X, Y)

```

1:  $n \leftarrow |X|, m \leftarrow |Y|$ 
2:  $maxLength \leftarrow 0, endIndex \leftarrow 0$ 
3:                                     ▷ 阶段 1: 扫描下三角及主对角线
4: for  $startX = 0$  to  $n - 1$  do
5:    $length \leftarrow 0$ 
6:    $i \leftarrow startX, j \leftarrow 0$ 
7:   while  $i < n$  AND  $j < m$  do
8:     if  $X[i] = Y[j]$  then
9:        $length \leftarrow length + 1$ 
10:      if  $length > maxLength$  then
11:         $maxLength \leftarrow length$ 
12:         $endIndex \leftarrow i$ 
13:      else
14:         $length \leftarrow 0$                                      ▷ 子串必须连续
15:         $i \leftarrow i + 1, j \leftarrow j + 1$ 
16:                                     ▷ 阶段 2: 扫描上三角（跳过主对角线）
17: for  $startY = 1$  to  $m - 1$  do
18:    $length \leftarrow 0$ 
19:    $i \leftarrow 0, j \leftarrow startY$ 
20:   while  $i < n$  AND  $j < m$  do
21:     if  $X[i] = Y[j]$  then
22:        $length \leftarrow length + 1$ 
23:       if  $length > maxLength$  then
24:          $maxLength \leftarrow length$ 
25:          $endIndex \leftarrow i$ 
26:       else
27:          $length \leftarrow 0$                                      ▷ 子串必须连续
28:          $i \leftarrow i + 1, j \leftarrow j + 1$ 
29: 从  $X[endIndex - maxLength + 1..endIndex]$  提取最长公共子串
30: return 最长公共子串及其长度  $maxLength$ 

```

时间复杂度分析：

关键观察：每个字符对 $(X[i], Y[j])$ 在整个算法执行过程中恰好被访问一次。

详细分析：

- 阶段 1 有 n 条对角线。第 k 条对角线（从 $X[k]$ 开始）的长度为 $\min(n - k, m)$ 。

所有对角线的总长度为：

$$\sum_{k=0}^{n-1} \min(n-k, m)$$

- 阶段 2 有 $m-1$ 条对角线。第 k 条对角线（从 $Y[k]$ 开始， $k \geq 1$ ）的长度为 $\min(n, m-k)$ 。所有对角线的总长度为：

$$\sum_{k=1}^{m-1} \min(n, m-k)$$

- 两阶段的对角线长度之和恰好等于矩阵中所有位置的数量，即 nm

因此，算法的字符比较次数恰好为 $\Theta(nm)$ ，每次比较的时间复杂度为 $O(1)$ ，总时间复杂度为 $\Theta(nm)$ 。

空间复杂度： $O(1)$ ，只使用常数个变量。

C++ 代码实现见：`code/longest_common_substring_non_dp.cpp`